



Pipeline de Análise de Dados Não Estruturados (UDA)

Automação de Relatórios de
Conjuntura do Setor Habitacional

Ingrid Soares

Projeto Individual 4 - Engenharia de Dados e IA

O Problema (Contextualização)



O Desafio dos Dados

80% a 90% dos dados globais são não estruturados (PDFs, relatórios)



Cenário Atual

O Ministério das Cidades precisa de dados trimestrais para o Relatório de Conjuntura



Gargalo

As informações estão pulverizadas em portais de RI em formatos variados



Risco

Métodos tradicionais (RegEx, coord. fixas) quebram se o PDF mudar o layout

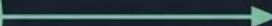
Objetivo do Projeto

Criar um pipeline inteligente, resiliente e automatizado.



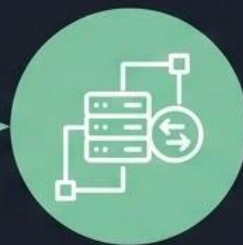
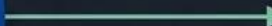
1. Coleta

Monitoramento automático de portais de RI.



2. Processamento

Extração semântica via LLMs (IA).



3. Entrega

Disponibilização de dados via API estruturada.

Arquitetura do Sistema (Visão Geral)



Camada de Extração: Ingestão automatizada e deduplicação.



Camada de Processamento (UDA): Segmentação de documentos e extração via LLM.



Camada de Persistência: Banco de dados com linhagem de dados (Data Lineage).



Camada de Serviço: API REST para consumo dos dados.

Ingestão Automatizada e Inteligente



Gatilhos de Ingestão:

Monitoramento de URLs de Centrais de Resultados.



Idempotência: Uso de Hashes (SHA-256) para evitar processar o mesmo PDF duas vezes.



Eficiência de Custo: Verificação no catálogo de dados antes de enviar o arquivo para a API do LLM (evitando gastos desnecessários de tokens).



Centrais de Resultados

Gatilhos/URLs



Processador Hashes (SHA-256)



Catálogo de Dados/Verificação



API do LLM

Evitando tokens desnecessários

Processamento UDA com LLMs

Abordagem Semântica: Substituição de regras rígidas por compreensão de contexto.

Estratégias de Leitura:



Full-Scan: Para documentos curtos.



Semantic Chunking: Divisão do PDF em blocos contextuais para otimizar tokens e precisão.

Motor de Extração:



Python (Parsing de PDF)



LLM API

Integração de Python - Parsing de PDF - LLM API.

Contrato Semântico e Qualidade

Estabelecimento de regras claras para a integridade dos dados extraídos.



Blindagem do Banco: Uso de esquemas estruturados (Pydantic/JSON Schema) para garantir que apenas dados válidos entrem no sistema.



Regras de Negócio: Inclusão de regras para forçar tipos de dados (Ex: Números, Datas) e tratamento de valores ausentes como NULL, evitando dados inconsistentes.



Foco em Valores Absolutos: A IA é instruída a ignorar 'marketing' (como porcentagens de variação) e extrair apenas os valores brutos reais (Ex: 'R\$ 100 mil' vs. 'crescimento de 10%').

Linhagem de Dados e API

Lineage e Rastreabilidade



Linhagem de Dados

- Cada dado no banco está vinculado ao link do PDF original.



Rastreabilidade

- Possibilidade de auditar a informação voltando ao documento fonte.

Camada de Serviço e Exemplo



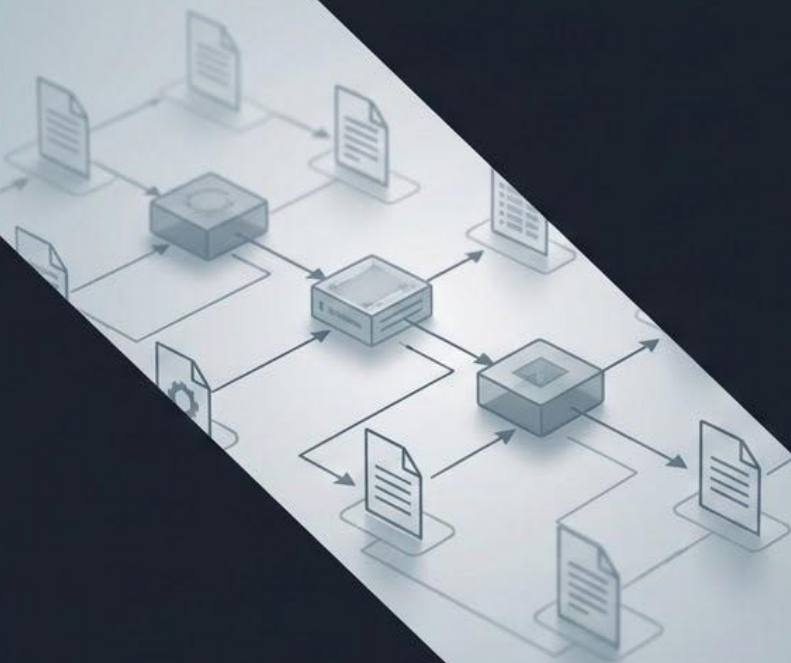
Camada de Serviço

- API REST simplificada.

Exemplo de Query (GET):

```
/api/conjuntura?empresa=MRV&ano=2025&trimestre=3`
```


Resiliência e Resultados



1



Teste de Layouts

O pipeline foi testado com diferentes formatos (Tabelas vs. Slides de Apresentação).

2



Resultado

Extração bem-sucedida independentemente da mudança visual do documento.

3



Vantagem

Manutenção zero ao mudar de empresa ou de versão de relatório.

Stack Tecnológica e Conclusão

Stack Tecnológica



Linguagem:
Python.



IA:
LLMs (via API).



Parsing:
PyMuPDF /
Bibliotecas de
extração de texto.



Banco de Dados:
SQLite/Postgres.

Conclusão



Sistema robusto,
escalável e adaptável a
dados não estruturados.



Project timeline

Fase 1:
Planejamento e
Análise

Fase 2:
Desenvolvimento
da Ingestão

Fase 3: Motor de
UDA e Prompt
Engineering

Fase 4:
Persistência e
Camada de
Serviço

Fase 5: Validação
e Testes de
Estresse

- Mapeamento dos portais de RI e análise da variabilidade dos PDFs.

- Implementação da coleta automática e sistema de deduplicação por hash.

- Criação do contrato semântico e integração com a API do LLM.

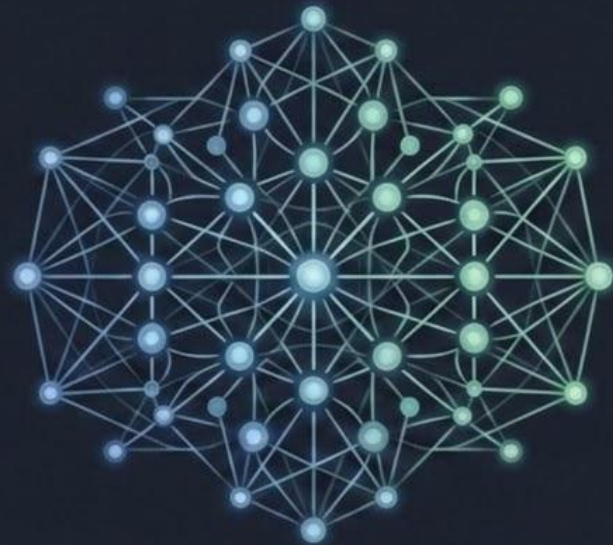
- Modelagem do banco de dados e desenvolvimento da API REST.

- Testes de resiliência com múltiplos layouts de construtoras diferentes.



Obrigada!

Github: @ingrdsoares



Referências:

1. AWS - What is Data Lineage?
(<https://aws.amazon.com/what-is/data-lineage/>)
2. Pinecone - Chunking Strategies for LLMs
(<https://www.pinecone.io/learn/chunking-strategies/>)